

JavaScript Functions – Notes & Examples

Pure Function

Definition: A pure function produces the same output for the same input and does not cause side effects. It does not modify external state such as global variables or input parameters.

```
function function_name() {  
    return;  
}
```

Anonymous Function

Definition: An anonymous function is a function declared without a name. It can be used where normal functions are used, especially with callback functions and event listeners.

```
var add = (function(a, b) {  
    c = a + b;  
    return c;  
})(5, 10);  
  
function trial() {  
    document.write(add + 1);  
}
```

Arrow Functions

Definition: Arrow functions are useful for writing short functions, especially single-expression functions. They are stored in variables and are easy to use.

```
// Arrow function to add two numbers  
const add = (a, b) => a + b;  
console.log(add(2, 3));  
  
// Arrow function with a single parameter  
const square = n => n * n;  
console.log(square(4));  
  
// Arrow function with multiple lines  
const greet = name => {  
    const message = `Hello, ${name}!`;   
    return message;  
};  
console.log(greet('mcet'));
```

Callback Function

Definition: A callback function is a function passed as an argument to another function and executed later, often after an operation is completed.

```
function fetchData(callback) {  
    setTimeout(() => {  
        const data = "Data fetched successfully!";  
        callback(data);  
    }, 2000);  
}  
  
fetchData(function(result) {  
    console.log(result);  
});
```

Higher-Order Function

Definition: A higher-order function is a function that takes one or more functions as arguments or returns a function as its result. It is commonly used for data transformation with methods such as map, filter, and reduce.

```
function applyFunction(arr, func) {  
    return arr.map(func);  
}  
  
// Function used as an argument  
function square(x) {  
    return x * x;  
}  
  
const numbers = [1, 2, 3, 4];  
const squaredNumbers = applyFunction(numbers, square);  
  
console.log(squaredNumbers);  
// Output: [1, 4, 9, 16]
```